

RAPPORT TECHNIQUE

Spring Bank Project

Application Bancaire Full-Stack

Module : Développement Web · Spring Boot & React

Année : 2025–2026

1. Introduction

Ce rapport présente le projet Spring Bank Project, une application bancaire full-stack développée dans le cadre du module de développement web. L'objectif est de mettre en pratique les concepts fondamentaux de Spring Boot pour le backend et de React pour le frontend, en implémentant les fonctionnalités essentielles d'un système de gestion bancaire simplifié.

L'application permet la gestion des clients, des comptes bancaires, des transactions et des virements, avec un système d'authentification différenciant les rôles ADMIN et CLIENT.

2. Architecture du Projet

2.1 Vue d'ensemble

Le projet suit une architecture client-serveur classique à deux niveaux distincts :

- Backend : API REST développée avec Spring Boot 3.3 (Java 21), exposant les données via des endpoints JSON
- Frontend : Application React (SPA) construite avec Vite, communiquant avec le backend via la bibliothèque Axios

Couche	Technologie	Rôle
Présentation	React + Vite + Axios	Interface utilisateur, SPA
API REST	Spring Boot 3.3 / Java 21	Contrôleurs HTTP, logique métier
Persistance	Spring Data JPA + H2	Entités, repositories, SQL
Build	Maven (backend) · npm (frontend)	Compilation, packaging

2.2 Structure des dossiers

- backend/src/main/java/com/bankapp/ — Code source Java (controllers, services, models, repositories, DTOs)
- backend/src/main/resources/ — Configuration Spring (application.properties) et données initiales (data.sql)
- frontend/src/ — Composants React, styles CSS et services API

3. Backend Spring Boot

3.1 Configuration

Le backend utilise Spring Boot 3.3.5 avec Java 21. La base de données H2 en mémoire est configurée dans application.properties avec initialisation automatique du schéma JPA (ddl-auto=create) et injection des données de test via data.sql au démarrage.

Les dépendances Maven incluent : spring-boot-starter-web, spring-boot-starter-data-jpa, h2 (runtime), lombok (optionnel) et spring-boot-starter-test.

3.2 Modèle de données

Quatre entités JPA constituent le cœur du domaine applicatif :

Entité	Attributs clés	Type	Relations
Client	id, username, phone	@Entity	1:1 AppUser, 1:N Account
AppUser	id, username, password, role	@Entity	1:1 Client
Account	id, accountNumber, balance	@Entity	N:1 Client, 1:N Transaction
Transaction	id, type, amount, date	@Entity	N:1 Account

Le type de transaction est modélisé par une enum TransactionType avec deux valeurs : CREDIT et DEBIT.

3.3 Couche Repository

Les repositories étendent JpaRepository et ajoutent des méthodes de recherche spécialisées :

- AccountRepository : findByAccountNumber(), findFirstByClientPhone(), findFirstByClientUsernameIgnoreCase() — permettant la résolution flexible du destinataire d'un virement
- TransactionRepository : findTop8ByOrderByDateDesc() pour les transactions récentes, findByIdOrderByDateDesc() pour l'historique par compte
- AppUserRepository : findByUsername() et existsByUsername() pour l'authentification

3.4 Couche Service

TransferService — Logique de virement

Le service de transfert est le composant métier le plus complexe. Il gère la résolution du destinataire (numéro de compte, téléphone ou username), valide le montant et le solde disponible, met à jour les deux comptes, puis enregistre les deux transactions DEBIT/CREDIT. L'opération est encapsulée dans une transaction @Transactional pour garantir l'atomicité.

AuthService — Authentification

L'authentification est implémentée de manière simplifiée sans JWT : vérification directe du mot de passe en clair, retour d'un AuthResponse contenant l'id, le username, le rôle et le clientId. L'inscription crée simultanément un Client et un AppUser via une transaction atomique.

AccountService & ClientService

Ces services exposent les opérations CRUD standard : findAll, create, update et delete, déléguant directement aux repositories correspondants.

3.5 Couche Controller & Endpoints

L'API REST est accessible sur le port 8080. Le CorsConfig autorise toutes les origines pour faciliter le développement local. Les endpoints disponibles sont :

Méthode	Endpoint	Contrôleur	Description
POST	/api/auth/login	AuthController	Authentification utilisateur
POST	/api/auth/register	AuthController	Inscription + création client
GET	/api/clients	ClientController	Liste des clients
POST	/api/clients	ClientController	Créer un client
GET	/api/accounts	AccountController	Liste des comptes
POST	/api/accounts	AccountController	Créer un compte bancaire
GET	/api/transactions/recent	TransactionController	8 dernières transactions
POST	/api/transfers	TransferController	Virement entre comptes

4. Frontend React

4.1 Technologie et organisation

Le frontend est une SPA (Single Page Application) construite avec React, Vite et lucide-react pour les icônes. L'ensemble de l'application est contenu dans un seul fichier App.jsx, avec les appels API centralisés dans services/api.js.

4.2 Gestion de l'état

L'état applicatif est géré localement avec les hooks React useState et useMemo. L'utilisateur connecté est persisté dans le localStorage pour survivre aux rechargements de page. Les données (comptes, clients, transactions) sont rechargées après chaque opération via la fonction loadData().

4.3 Système d'authentification frontend

La page de connexion propose deux modes : login et register. Après authentification, les données de session (username, role, clientId) sont stockées dans le localStorage. Le rôle conditionne l'affichage : un ADMIN voit tous les comptes et clients, peut créer des clients et des comptes ; un CLIENT ne voit que ses propres données.

4.4 Fonctionnalités de l'interface

- Dashboard avec 4 statistiques : nombre de comptes, de clients, de transactions récentes et solde global
- Formulaire de virement : sélection du compte source, saisie du destinataire (numéro, téléphone ou username) et du montant
- Panneau admin : création de clients et de comptes (réservé au rôle ADMIN)
- Liste des comptes bancaires en cartes avec solde affiché en MAD
- Historique des 8 dernières transactions avec code couleur DEBIT/CREDIT

5. Données de Test

Le fichier data.sql initialise la base avec trois profils de test permettant de valider tous les cas d'usage :

- admin / admin123 — rôle ADMIN, compte ACC-ADMIN-001 avec 10 000 MAD
- toufik / 123456 — rôle CLIENT, compte ACC-TOUFIK-001 avec 5 000 MAD
- mehdi / 123456 — rôle CLIENT, compte ACC-MEHDI-001 avec 2 600 MAD

Les séquences d'ID sont réinitialisées à 4 après l'insertion pour éviter les conflits lors des créations ultérieures.

6. Points Forts et Limites

6.1 Points forts

- Architecture claire respectant la séparation des couches (MVC + Service + Repository)
- TransferService robuste avec résolution multi-critères du destinataire et validation complète
- Inscription atomique créant automatiquement le Client associé à l'AppUser
- Interface responsive avec gestion des rôles et filtrage côté frontend

- Configuration CORS permissive adaptée au développement local

6.2 Limites et améliorations possibles

- Sécurité : les mots de passe sont stockés en clair — il faudrait utiliser BCryptPasswordEncoder
- Authentification : absence de JWT ou de session Spring Security — tout endpoint est accessible sans token
- Pas de gestion globale des exceptions (ExceptionHandler) : les erreurs renvoient des 500 non structurés
- Base H2 en mémoire : les données sont perdues à chaque redémarrage — migration vers PostgreSQL/MySQL recommandée
- Frontend monolithique : App.jsx (400+ lignes) gagnerait à être découpé en composants réutilisables
- Absence de tests unitaires pour les services métier

7. Lancement de l'Application

7.1 Backend

Prérequis : Java 21, Maven. Commandes de démarrage :

- `cd backend && ./mvnw spring-boot:run`
- L'API est disponible sur `http://localhost:8080`
- La console H2 est accessible sur `http://localhost:8080/h2-console` (JDBC URL : `jdbc:h2:mem:bankdb`)

7.2 Frontend

Prérequis : Node.js, npm. Commandes de démarrage :

- `cd frontend && npm install && npm run dev`
- L'interface est disponible sur `http://localhost:5173`

8. Conclusion

Le projet Spring Bank Project constitue une implémentation fonctionnelle et bien structurée d'une application bancaire simplifiée. Il démontre la maîtrise des concepts fondamentaux de Spring Boot (JPA, repositories, services transactionnels, REST controllers) et de React (état local, hooks, routing conditionnel par rôle).

Les principales pistes d'amélioration concernent la sécurité (chiffrement des mots de passe, Spring Security avec JWT), la robustesse (gestion globale des erreurs, base de données persistante) et la maintenabilité du code frontend (découpage en composants). Ces évolutions permettraient de passer d'un prototype pédagogique à une application proche des standards de production.